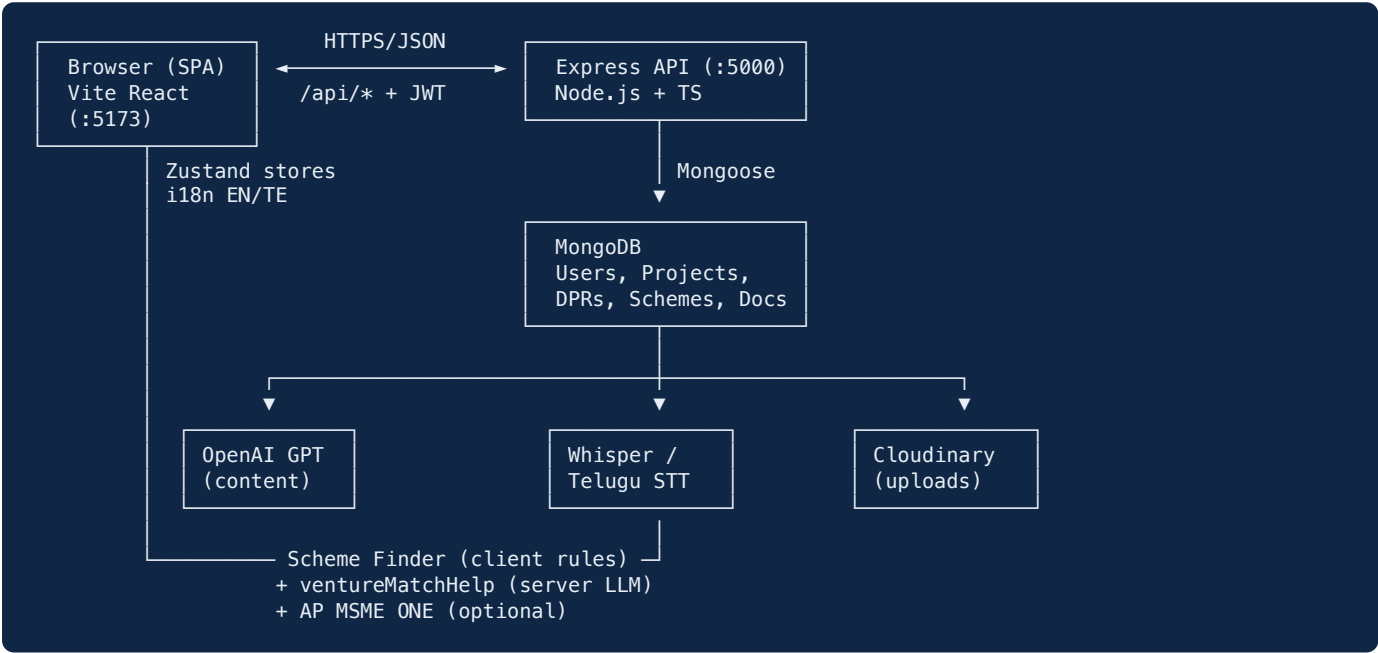


Application Architecture

Repository: **AI_DRP** · Stack: React + Vite / Express + MongoDB · Generated from codebase structure · Bilingual EN / TE

Purpose. Guide entrepreneurs to create bank-ready Detailed Project Reports (DPRs), match them to central and Andhra Pradesh MSME schemes, and support AI-assisted drafting, analytics, and admin document workflows — with optional AP MSME ONE portal integration.

1. System context



2. Repository layout

PATH	ROLE
client/	React 18 SPA (Vite, Tailwind, Zustand, react-i18next, Recharts)
server/	Express API, Mongo models, AI/PDF/scheme services, JWT auth
docs/	Architecture and product notes (this PDF)

Client (`client/src`)

- `pages/` — routes / screens
- `components/` — UI by domain
- `store/` — Zustand state
- `lib/` — API client, scheme matcher, DPR helpers
- `i18n/` — EN / TE locale packs

Server (`server/src`)

- `routes/` — HTTP mount points
- `controllers/` — request handlers
- `services/` — business + AI logic
- `models/` — Mongoose schemas
- `middleware/` — auth, errors, rate limits

3. Frontend routes & product surfaces

ROUTE	PAGE	WHAT IT DOES
<code>/</code>	Landing	Marketing / entry
<code>/login</code> , <code>/register</code>	Auth	JWT session; roles: entrepreneur, officer, admin
<code>/dashboard</code>	Dashboard	User home / shortcuts
<code>/projects</code>	Projects	CRUD project shells for DPRs
<code>/venture-match</code>	Scheme Finder	14-question wizard → ranked scheme matches + CMEP booster
<code>/individual-dpr/create</code>	Individual Latest DPR	18-step unit DPR; scheme overlay + scheme brief panel
<code>/cluster-dpr/create</code>	Cluster DPR	Multi-unit / SPV-style cluster report flow
<code>/dpr/builder</code>	AI Guided Builder	Chat + step analyze assist for classic DPR sessions
<code>/dpr/:projectId</code> , <code>/dpr/view/:dprId</code>	Generate / Preview	Generation and document view / download
<code>/chat</code>	Chat	General AI guidance chatbot
<code>/admin</code> , <code>/admin/documents</code>	Admin	Ops dashboard + document / template management
<code>/profile</code>	Profile	User profile / Udyam / contact

Protected routes wrap authenticated pages via `ProtectedRoute` . Language toggle drives EN/TE UI strings and DPR view language.

4. Client component domains

FOLDER	RESPONSIBILITY
components/venture-match/	Question cards, results, help bubble, step exclusions
components/individual-dpr/	Step form, scheme brief panel (Benefits / Eligibility / ...)
components/cluster-dpr/	Cluster form, AI suggestions, document view, financials, charts
components/dpr-builder/	Step analyze assist + step chat panel
components/dpr/	Legacy / shared DPR workflow widgets
components/layout/ , ui/ , auth/	Shell, primitives, role picker, route guard

Zustand stores

- authStore — user, token, role
- projectStore — project list / selection
- dprStore / chatStore — classic session + chat state
- clusterDPRStore — cluster multi-step draft
- individualDPRStore — individual steps, matched scheme, VentureMatch answers, extras

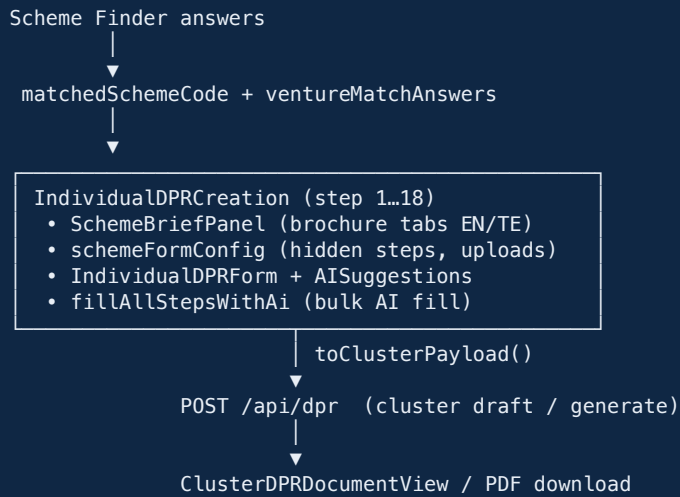
5. Scheme Finder (client-first matching)

Matching logic lives primarily on the **client** under `client/src/lib/ventureMatch/`. The server adds an LLM helper for clarifying free-text answers (`ventureMatchHelp.service`).

MODULE	ROLE
questions.ts / types.ts	14 cards + option IDs (activity includes <code>knowledge</code> ; owners include transgender / ex-serviceman)
schemes.ts	16 scheme rules (15 original + <code>AP_CMEP</code>); pass/fail/unknown criteria; CMEP booster
evaluate.ts	Scores remaining schemes each step; exclusion reasons
helpGuides.ts	AP-oriented chip trees for hybrid help chat
mapToDpr.ts	Handoff → Individual DPR prefill + <code>?scheme=</code>

AP domicile gate applies only to AP state schemes (EDP, FPP, Tech Upgrade, Parks, OBMMS, CMEP). Central schemes remain available to AP residents.

6. Individual Latest DPR pipeline



Scheme briefs are static bilingual packs in `lib/individualDpr/schemeBriefs/` (central + AP), rendered like AP MSME One (Benefits, Eligibility, How to Apply, Documents, FAQs + Quick Info).

7. Backend API map

All business routes mount under `/api` . Health: `GET /api/health` .

MOUNT	CONCERN
<code>/api/auth</code>	Register, login, JWT profile
<code>/api/projects</code>	Project CRUD
<code>/api/dpr</code>	Sessions, builder analyze/chat, uploads, cluster individual drafts, PDF/DOCX, generation
<code>/api/schemes</code>	Scheme catalog / matching helpers
<code>/api/ai</code>	Chat, suggestions, VentureMatch help, translation-related AI
<code>/api/apmsme</code>	AP MSME ONE bridge (schemes, submit, status)
<code>/api/documents</code>	Admin / knowledge documents
<code>/api/feedback</code>	User feedback for quality loops
<code>/api/admin</code>	Admin analytics & ops

Cross-cutting middleware

- Helmet, CORS, compression, Morgan logging
- JSON body up to 50mb (HTML capture for print-faithful PDFs)
- Rate limiter on `/api`
- `authenticate` JWT guard on protected controllers
- Central `errorHandler` / `notFound`
- Static `/uploads` with CORS for embedded PDFs/images

8. Core services

SERVICE	RESPONSIBILITY
<code>openai.service</code>	Primary LLM for DPR content, chat, field suggestions
<code>gemini.service</code>	Alternate / supplemental generative path
<code>dpr.service</code> / <code>clusterDPR.service</code>	Session orchestration, cluster/individual draft persistence, generation
<code>dprBuilderAssist.service</code>	Grounded step analyze + step chat for AI Guided Builder
<code>financial.service</code>	Cost structures, viability helpers
<code>scheme.service</code>	Scheme data / recommendation support
<code>ventureMatchHelp.service</code>	LLM fallback for Scheme Finder free-text help
<code>quality.service</code> / <code>ml.service</code>	Quality scores, bankability / analytics signals
<code>translation.service</code>	EN ↔ TE content support

SERVICE	RESPONSIBILITY
whisper-telugu.service	Speech-to-text (Telugu-aware)
cloudinary.service	Media / file hosting when configured
apmsme.service	External AP MSME ONE API adapter (+ mocks)
suggestion.service	Field-level AI suggestions

9. Data model (MongoDB)

MODEL	STORES
User	Identity, bcrypt password, role (entrepreneur / officer / admin), Udyam, location
Project	Entrepreneur project shell linked to DPRs
DPRSession / versions / templates	Guided builder sessions, templates, version history
ClusterSection	Cluster / individual multi-step draft sections
Scheme / SchemeMatch	Scheme catalog and match records
Document / Policy	Admin knowledge / policy docs
Feedback / DPRAnalytics / RAGMetrics	Feedback loops and analytics
VectorStore	Embeddings / RAG support when used

10. End-to-end flows

A. Scheme → Individual DPR

1. User answers Scheme Finder questions; client evaluator narrows 16 schemes.
2. Optional help bubble: chip tree first, then LLM via `/api/ai`.
3. “Generate DPR for this Match” writes handoff + navigates to Individual DPR with scheme code.
4. Step 1 shows scheme brief; form fields / uploads adapt via `schemeFormConfig`.
5. Drafts save through cluster DPR APIs; generate produces bank-ready document (EN/TE).

B. Cluster DPR

1. User fills cluster steps in `ClusterDPRCreation` + shared AI suggestion widgets.
2. Server `clusterDPR.service` persists sections and generates narrative / tables.
3. Preview via `ClusterDPRDocumentView`; export PDF (incl. HTML capture path).

C. AI Guided Builder

1. Session started under `/api/dpr/sessions`.
2. Per-step analyze (`/builder/analyze`) and chat (`/builder/chat`) grounded on prompts in `constants/`.
3. Final DPR downloadable as PDF/DOCX from session endpoints.

11. Cross-cutting product capabilities

- **Bilingual:** react-i18next locales + server translation service; DPR view language switch.
- **Offline / mock:** `offlineDetector`, `mockData`, offline QA / DPR helpers for degraded networks.
- **RBAC:** client `rbac.ts` + server role on User; admin routes gated.
- **Quality loop:** feedback + ML/quality services → admin analytics.
- **Deploy:** client/server `vercel.json`; Railway references in helmet CSP for hosted API.

12. Architecture principles (as implemented)

- **SPA + API:** UI never talks to Mongo directly; all persistence through Express.
- **Domain packs on client:** Scheme matching and scheme brochure copy ship with the SPA for snappy UX; LLM used when rules need language understanding.
- **Shared DPR document view:** Individual flow reuses cluster document rendering via payload adapter.
- **Scheme-aware forms:** One 18-step skeleton; scheme code toggles fields, hidden steps, and uploads.
- **AI as co-pilot:** Suggestions and fill-all assist; human review before generate.

13. Local runtime

PROCESS	DEFAULT
Client Vite	<code>http://localhost:5173</code> — <code>VITE_API_URL</code> → API
Server Express	<code>http://localhost:5000/api</code> — requires <code>OPENAI_API_KEY</code> , Mongo URI

AI_DRP application architecture — derived from repository layout, routes, stores, and services. Not a formal compliance document; scheme percentages and GOs should be confirmed with implementing agencies.